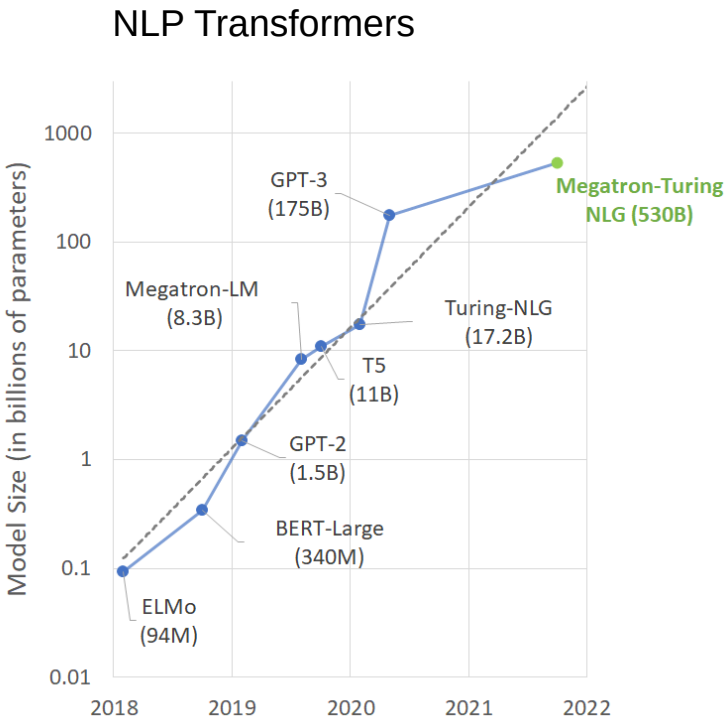
Optimizing the learning of very large models

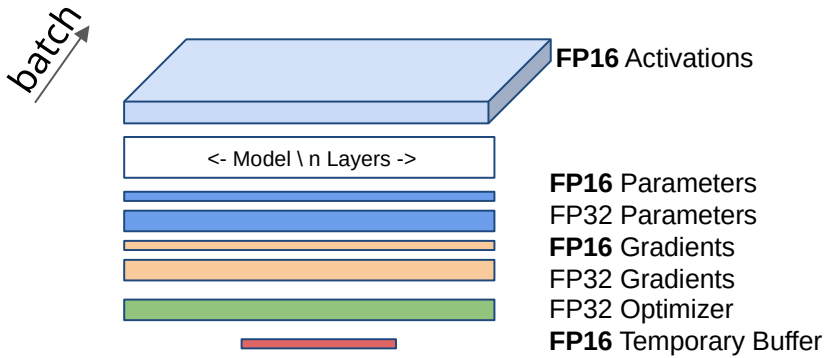
Florian SABY

Large model > 1 billion parameters





CUDA Out Of Memory



[1]

Python libraries for efficient LLM serving





[1]

Optimizing the learning of very large models

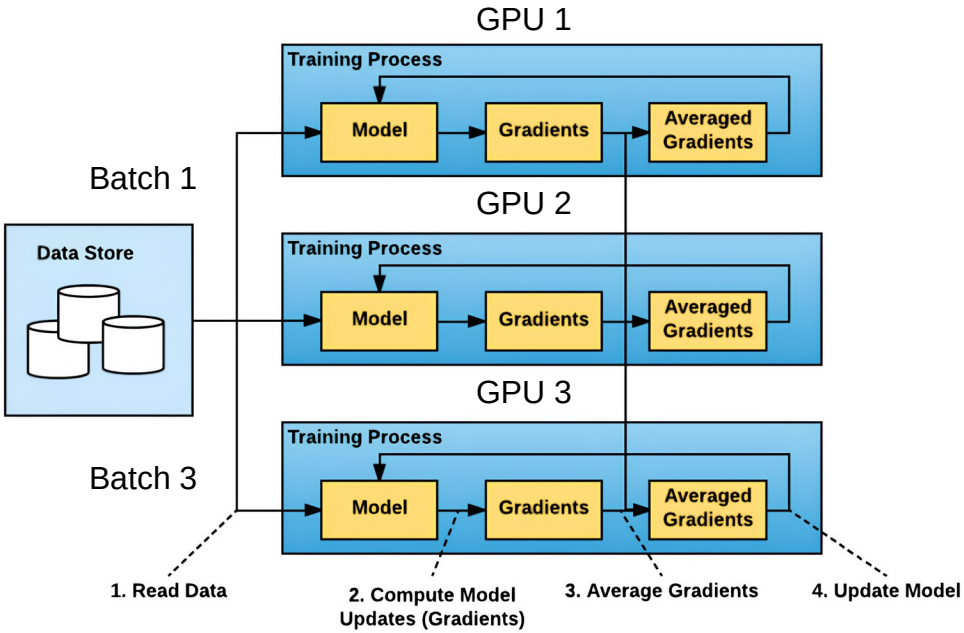
Data parallelism ◂

Pipeline parallelism ◂

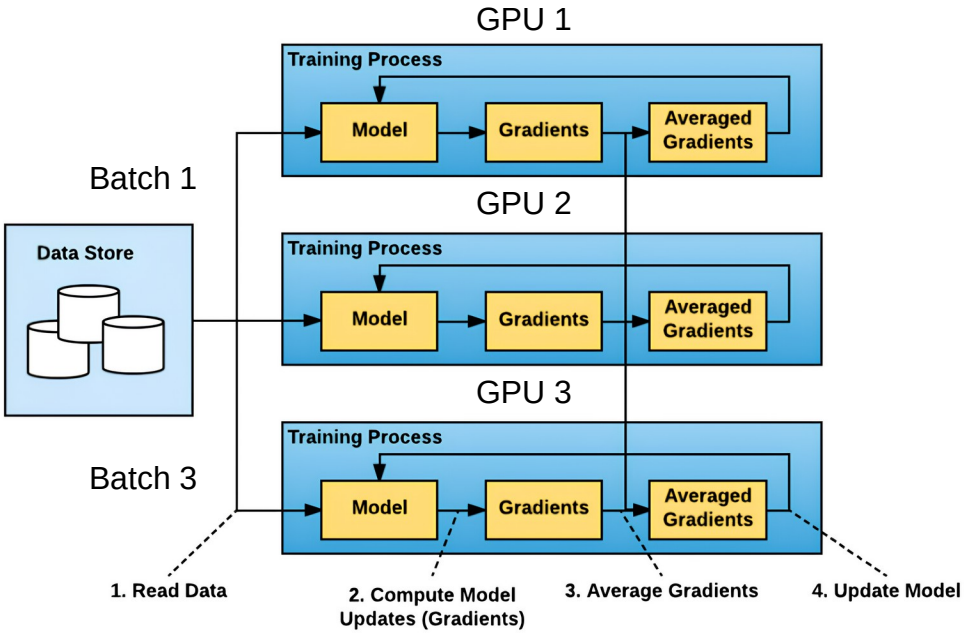
Tensor parallelism ◂

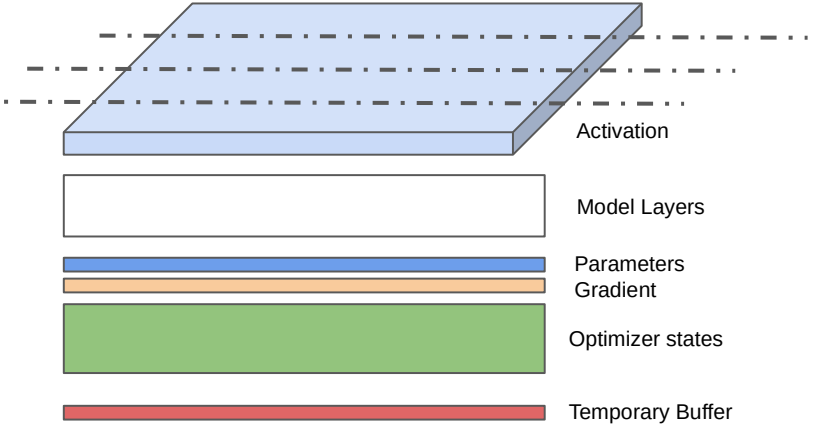
3D parallelism ◂

Data Parallelism

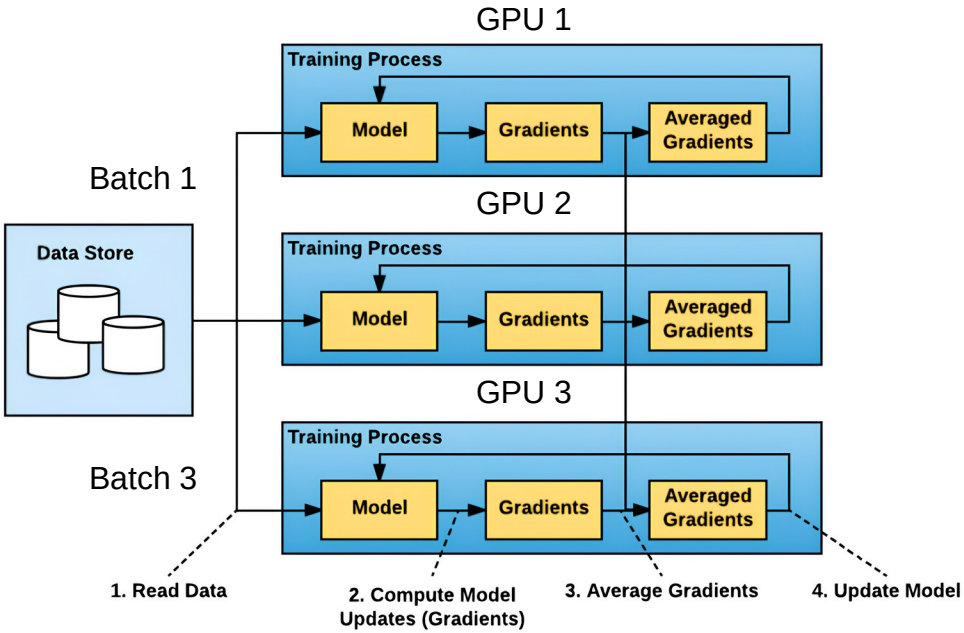


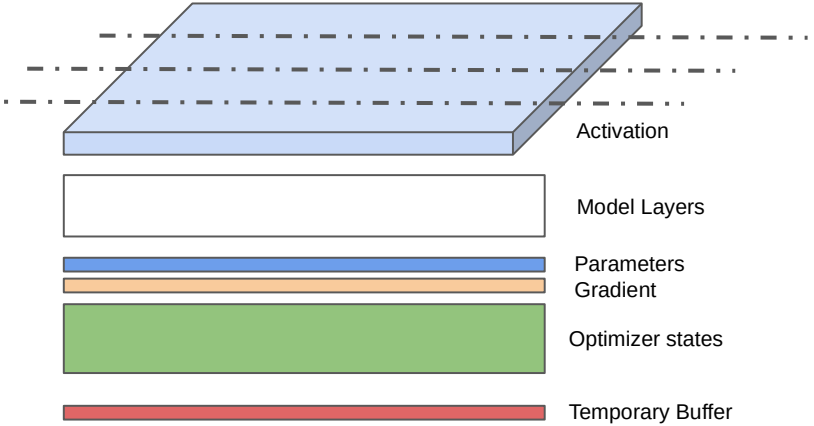
Data Parallelism





Data Parallelism

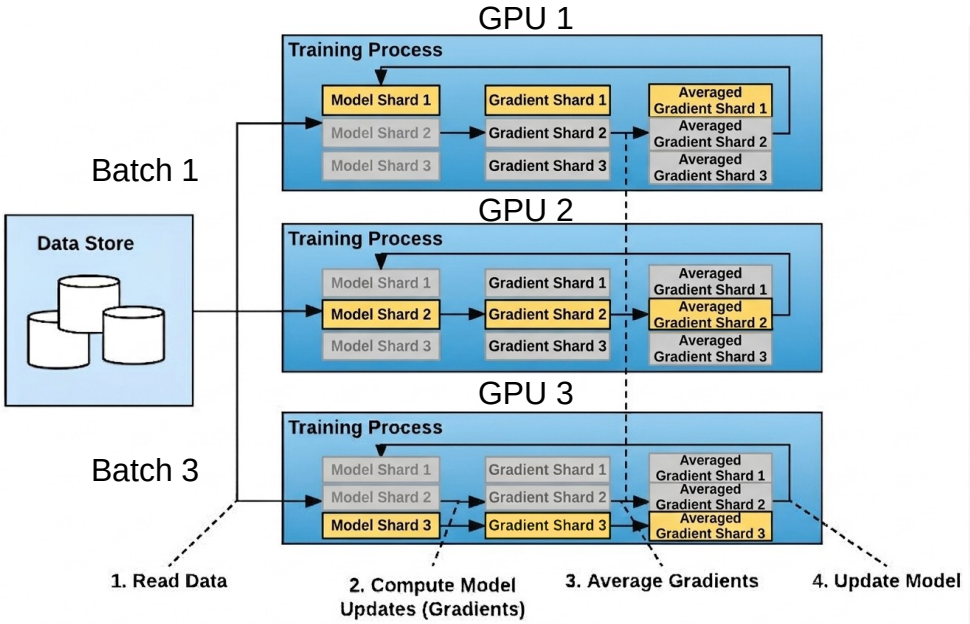




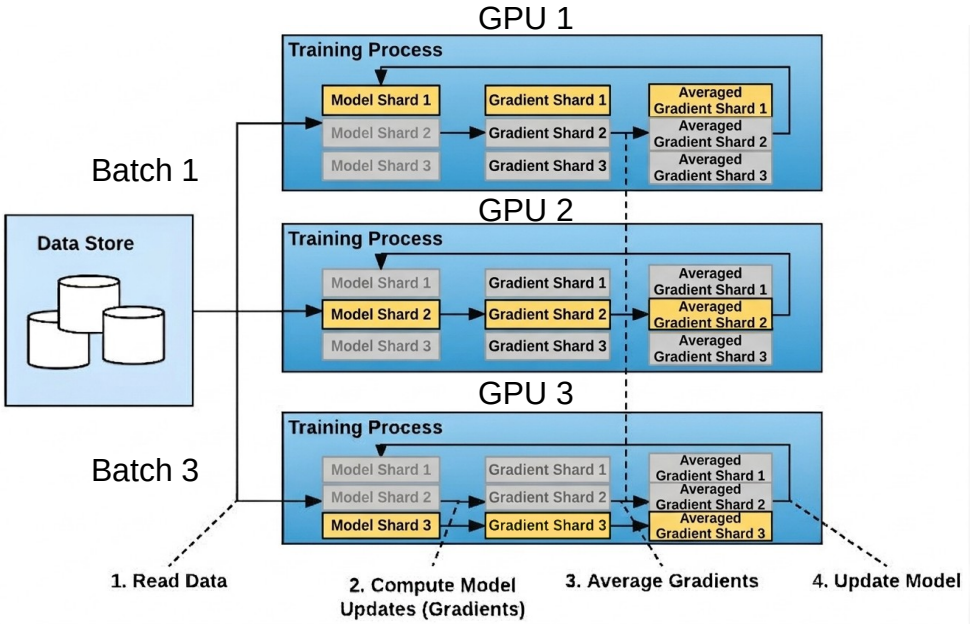
Advantages
-High Throughput
-Only the activation memory footprint is distributed
-Multi-processing

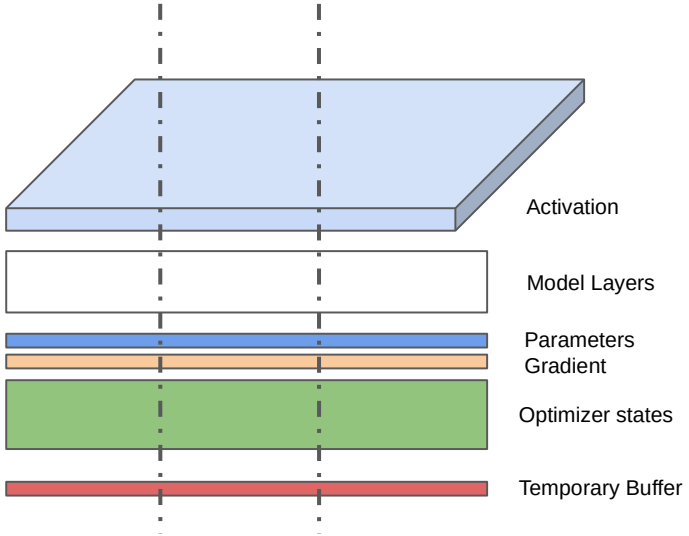
Drawback
-Each GPU must be able to fit the full model in VRAM.

Fully Sharded Data Parallelism

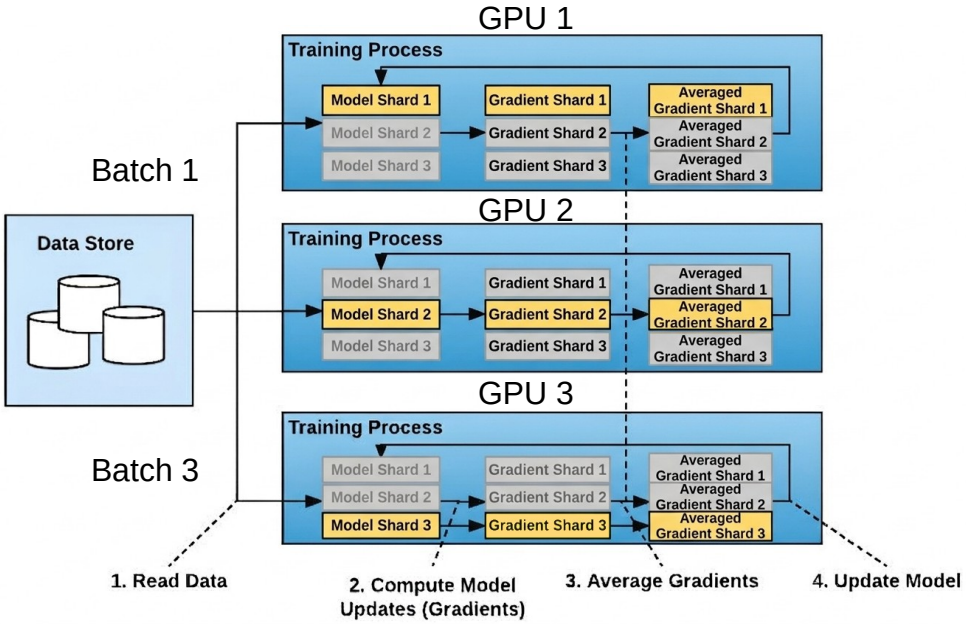


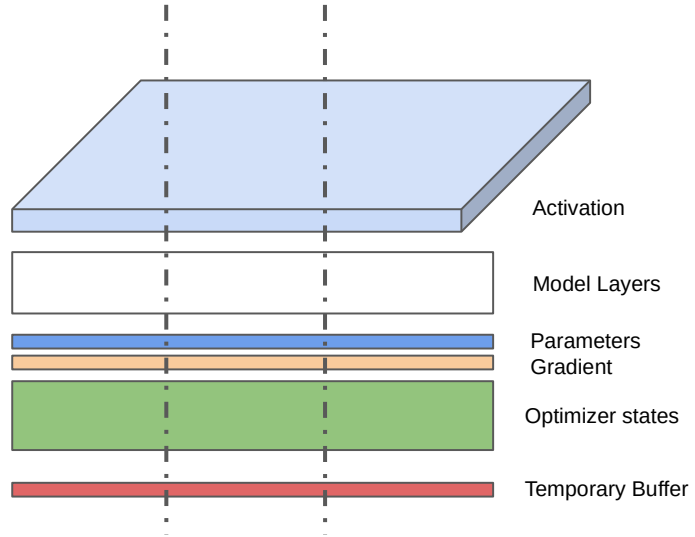
Fully Sharded Data Parallelism





Fully Sharded Data Parallelism

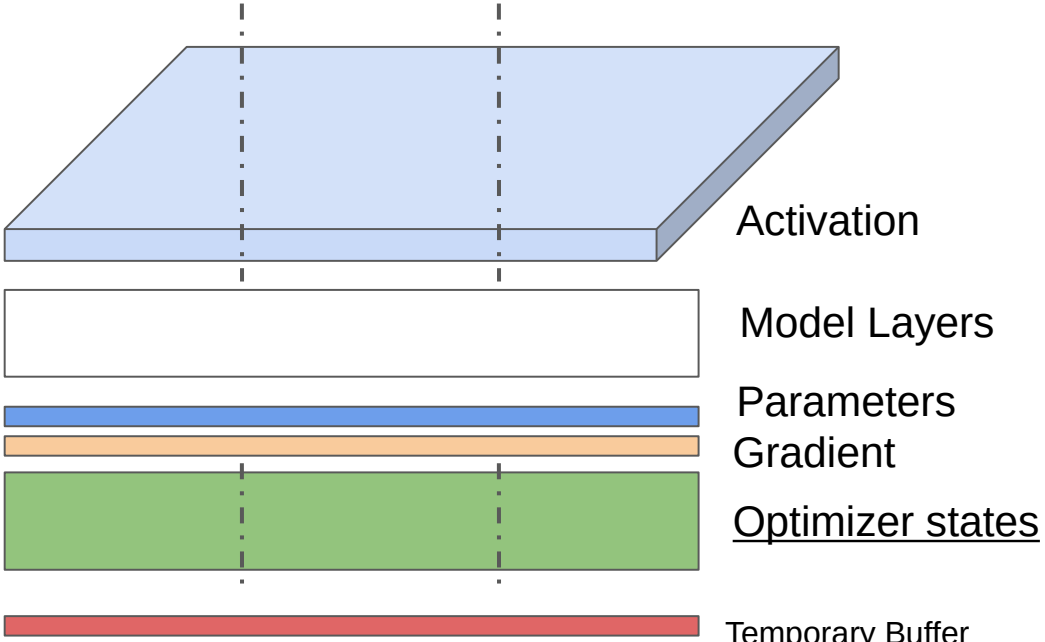




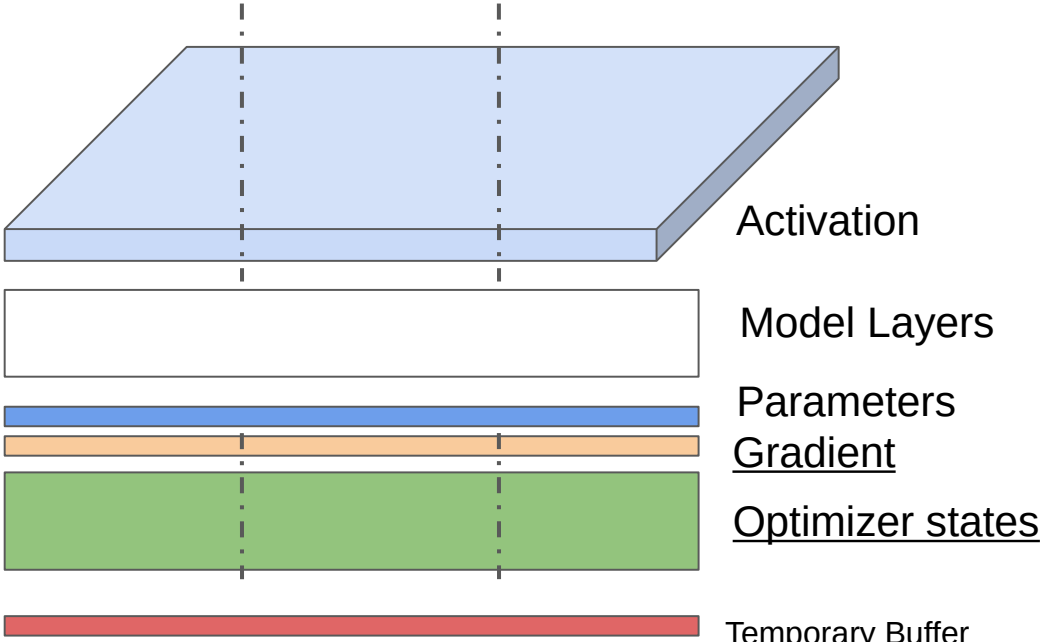
Advantage
-Lets you run models larger than your VRAM

Drawback
-FSDP generates a lot of communication

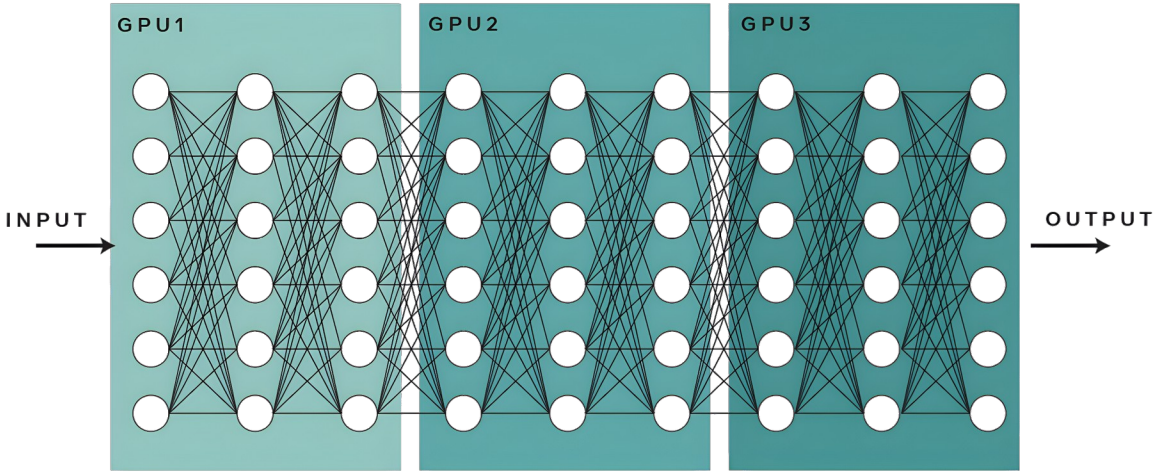
Alternative to FSDP : ZERO 1



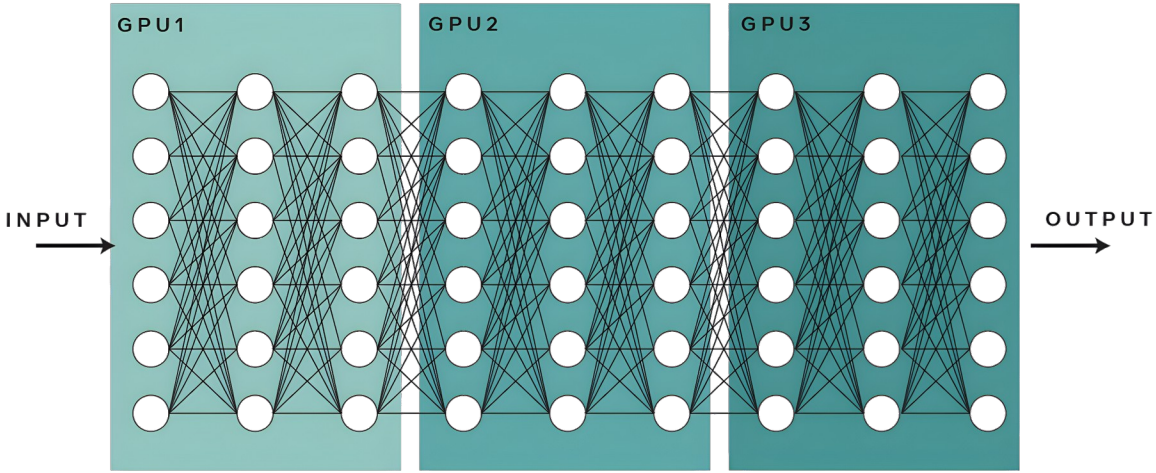
Alternative to FSDP : ZERO 1

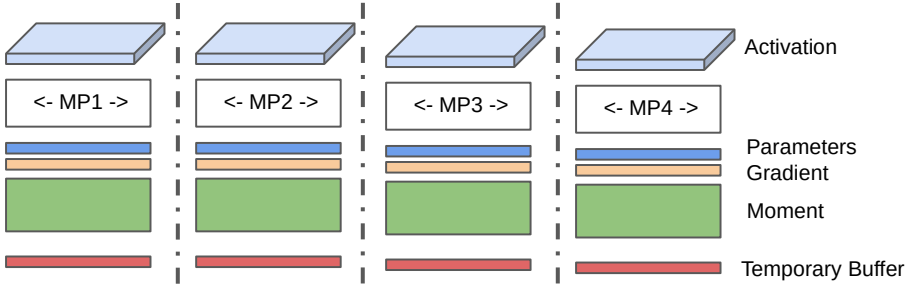


Pipeline Parallelism



Pipeline Parallelism

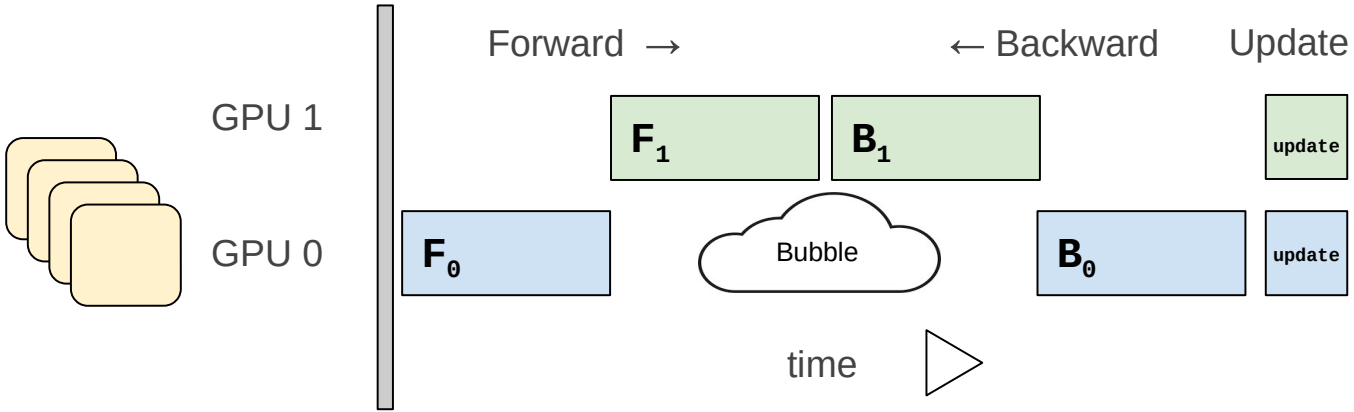




Advantage

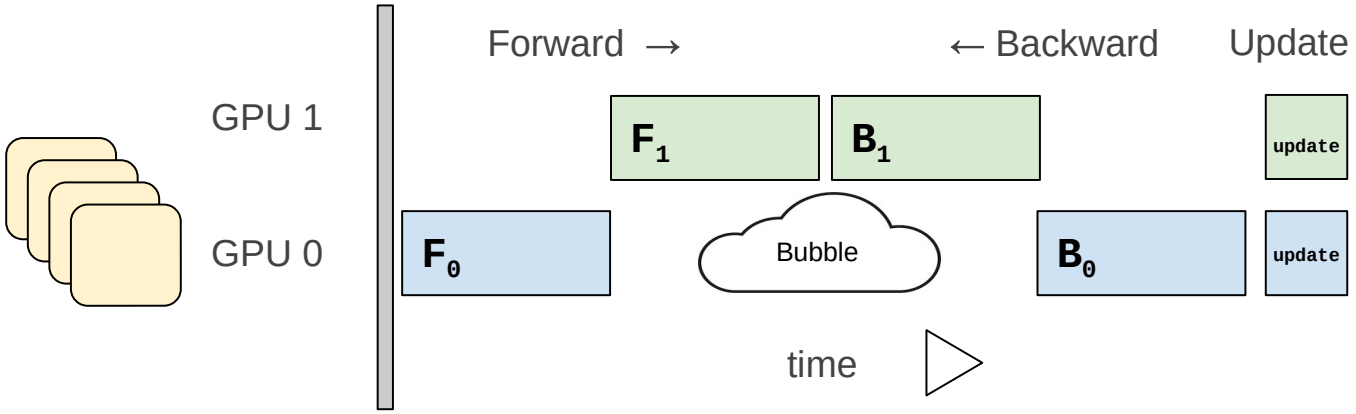
- Distributed memory footprint
- Low communication overhead
- No need to wait before sending the next batch

PP : Micro-batching

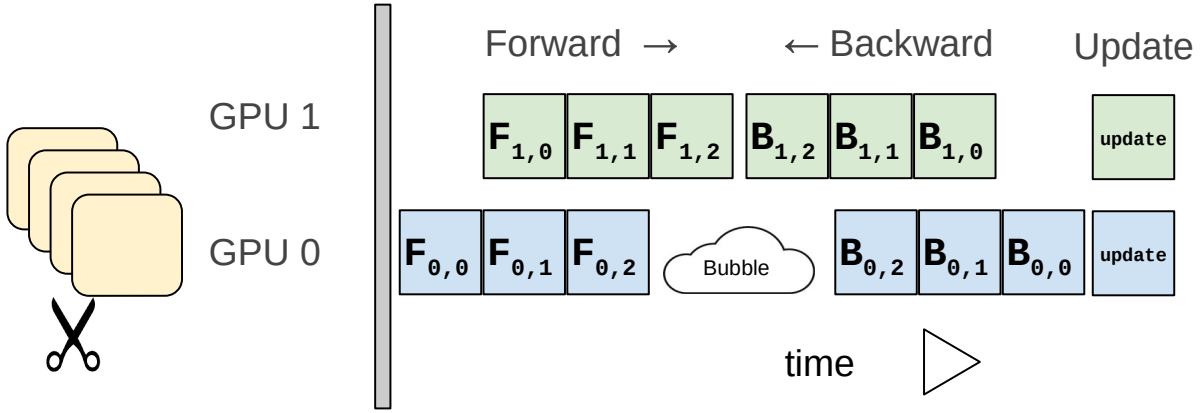


Conventional batching

PP : Micro-batching



Conventional batching

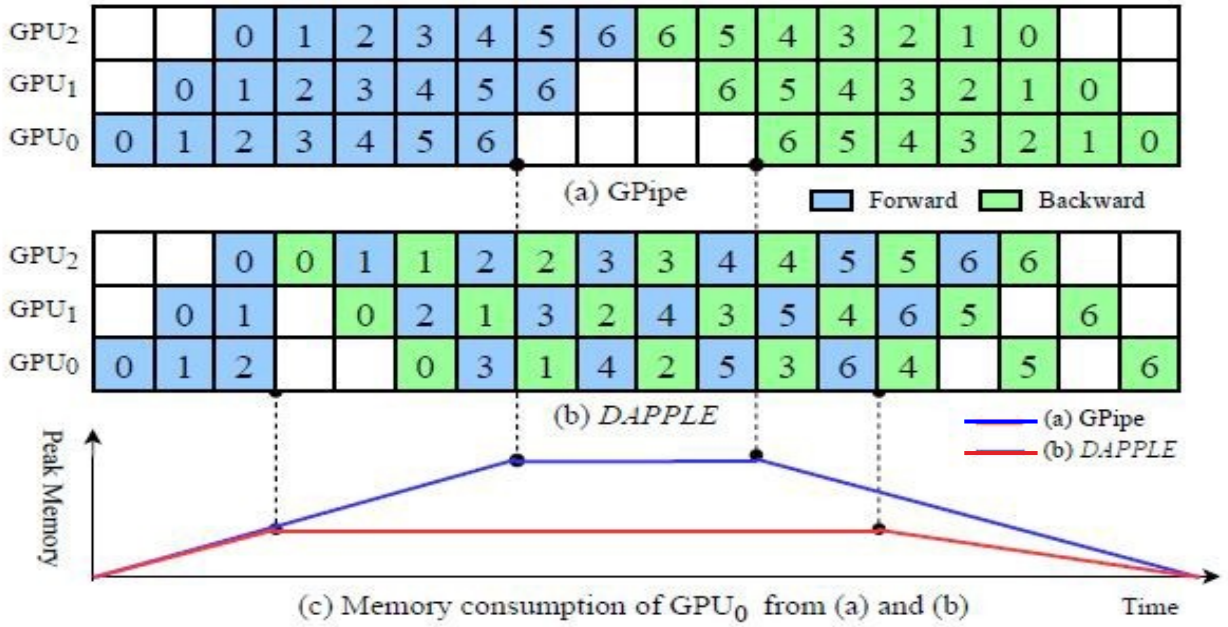


Micro batching

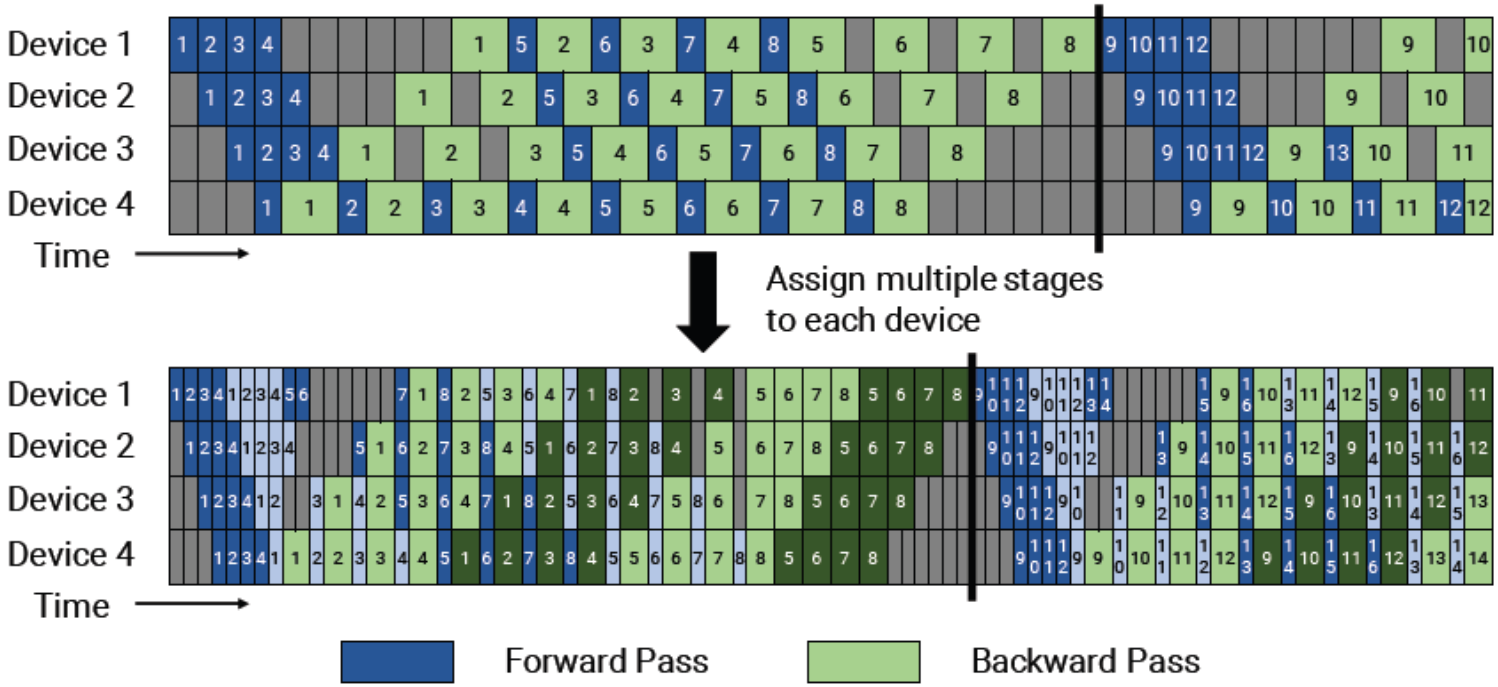
PP : Optimization with micro-batching

All Forward All Backward

One Forward One Backward



PP : Interleaved 1F1B



We split each pipeline stage into smaller blocks

PP : Zero Bubble

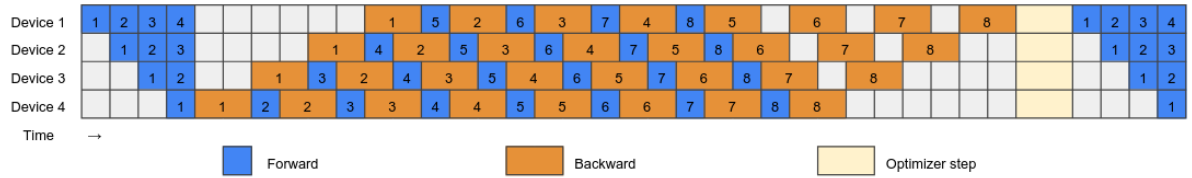


Figure 2: 1F1B pipeline schedule.

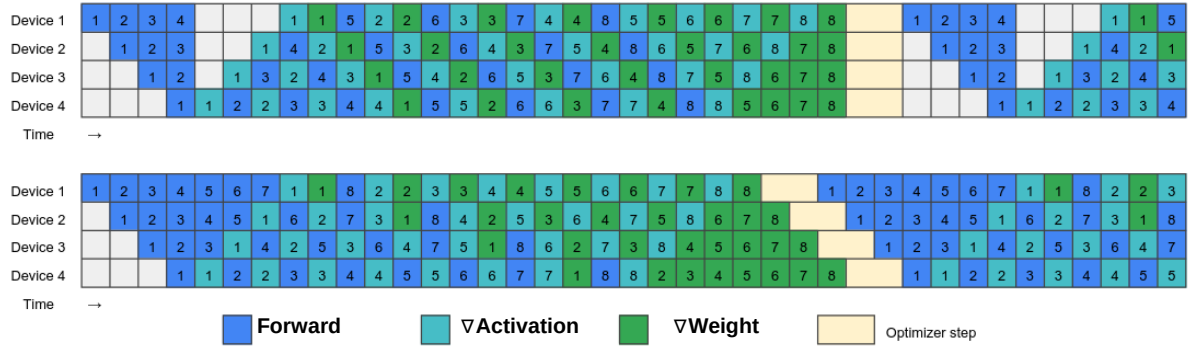
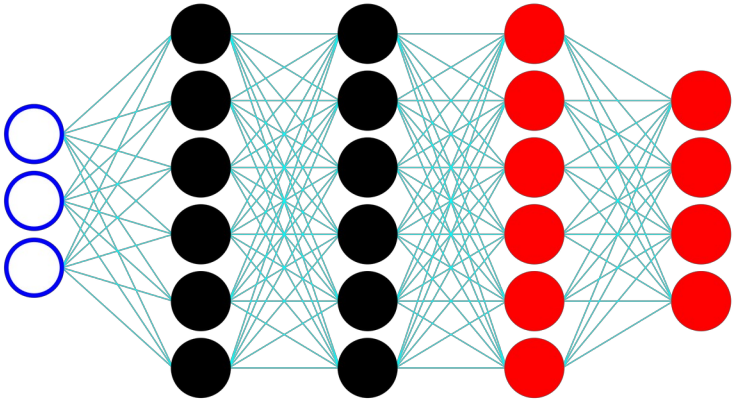


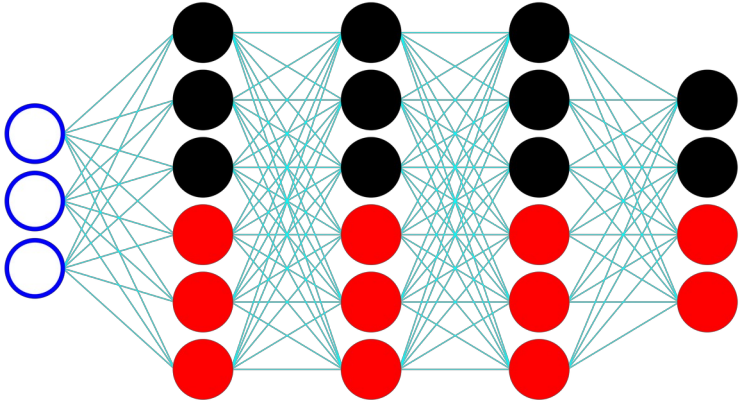
Figure 3: Handcrafted pipeline schedules, top: ZB-H1; bottom: ZB-H2

Two paradigms for model parallelism

Pipeline Parallelism



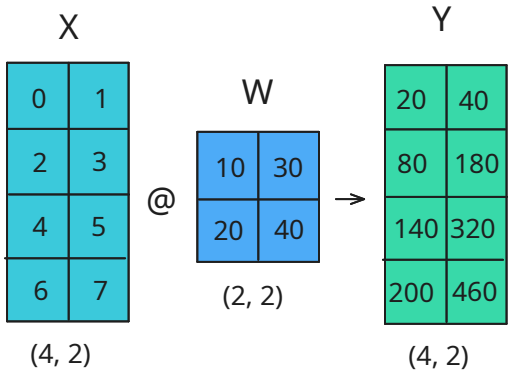
Tensor Parallelism

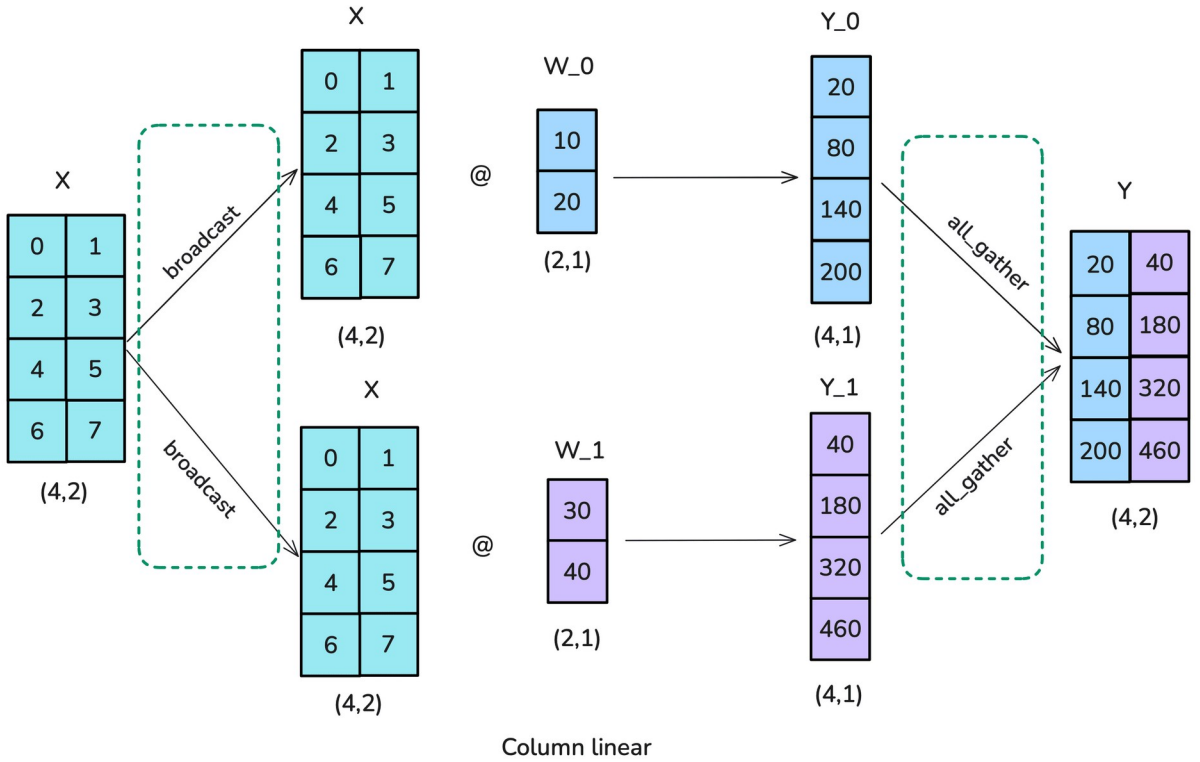


GPU 0

GPU 1

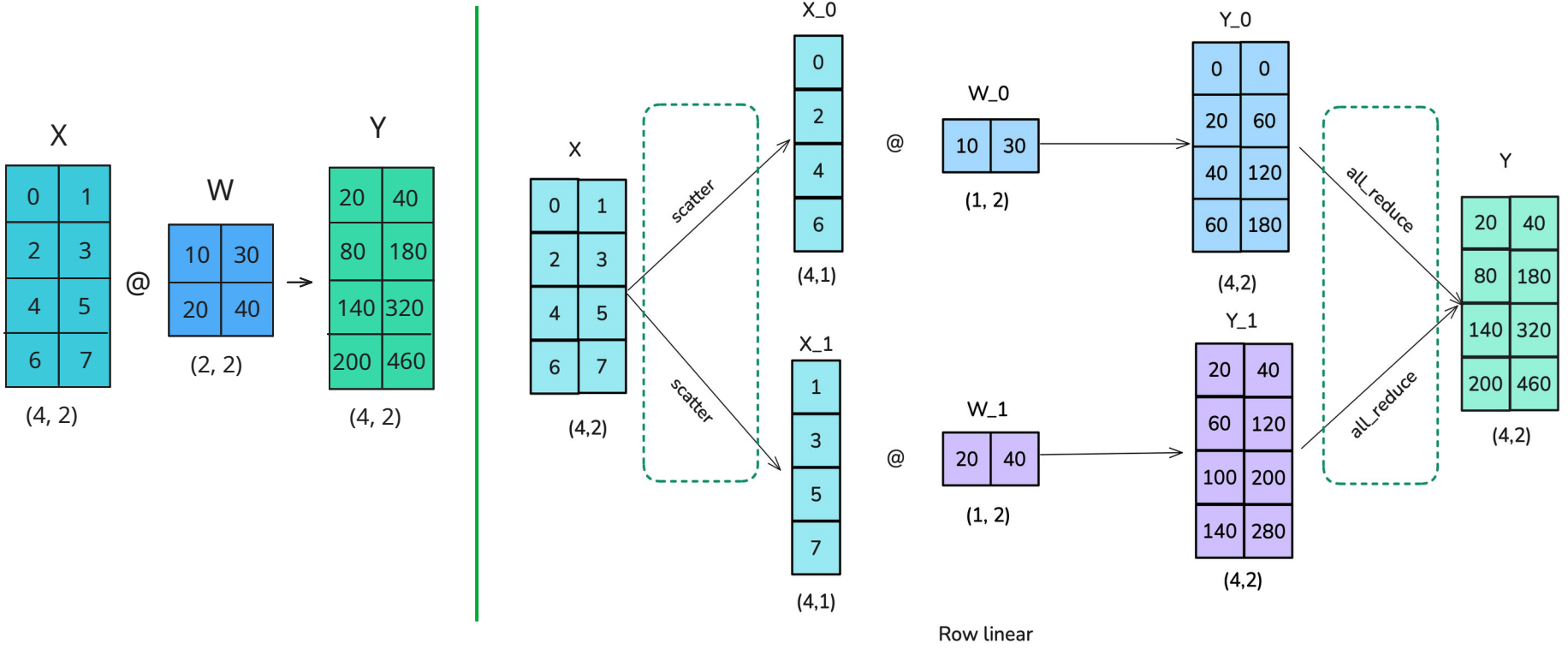
TP : Columns





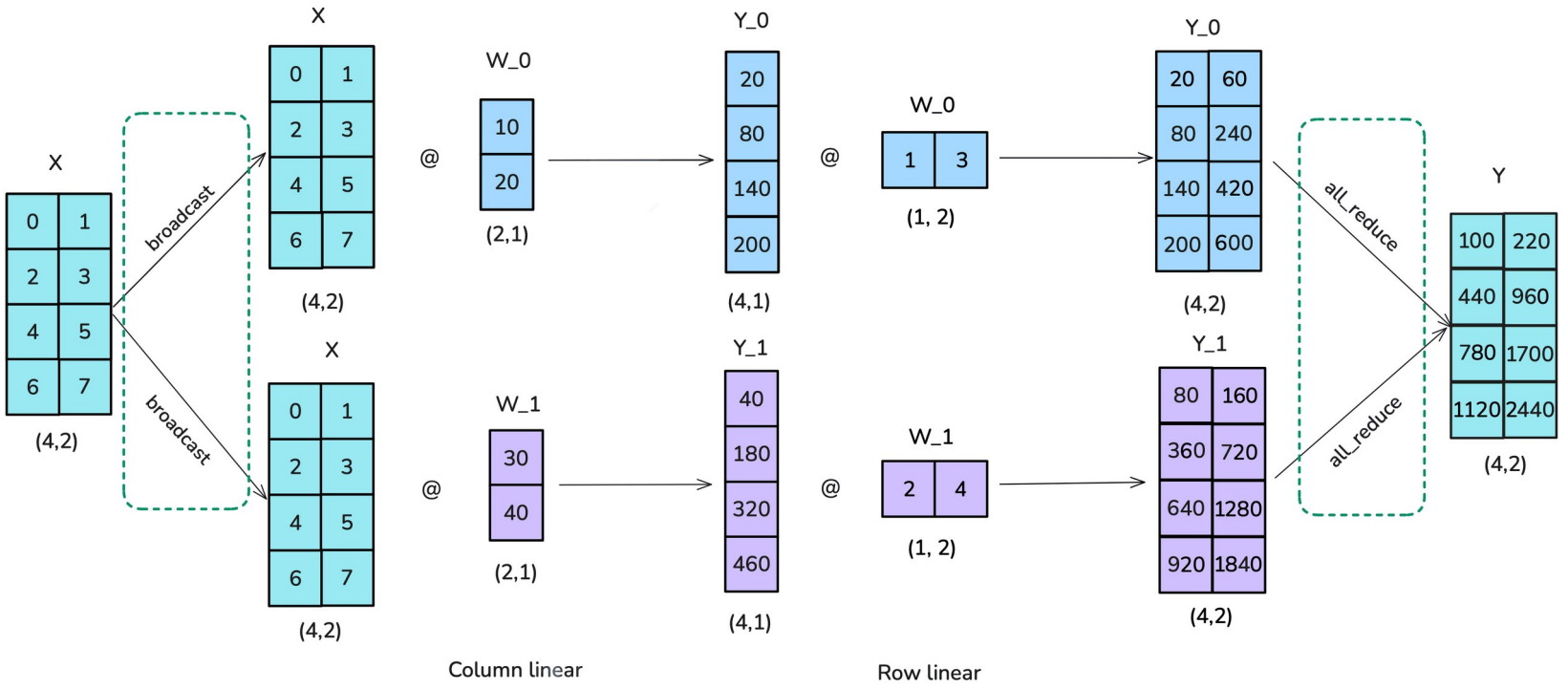
Drawback: Tensor parallelism requires synchronization per layer

TP : Rows



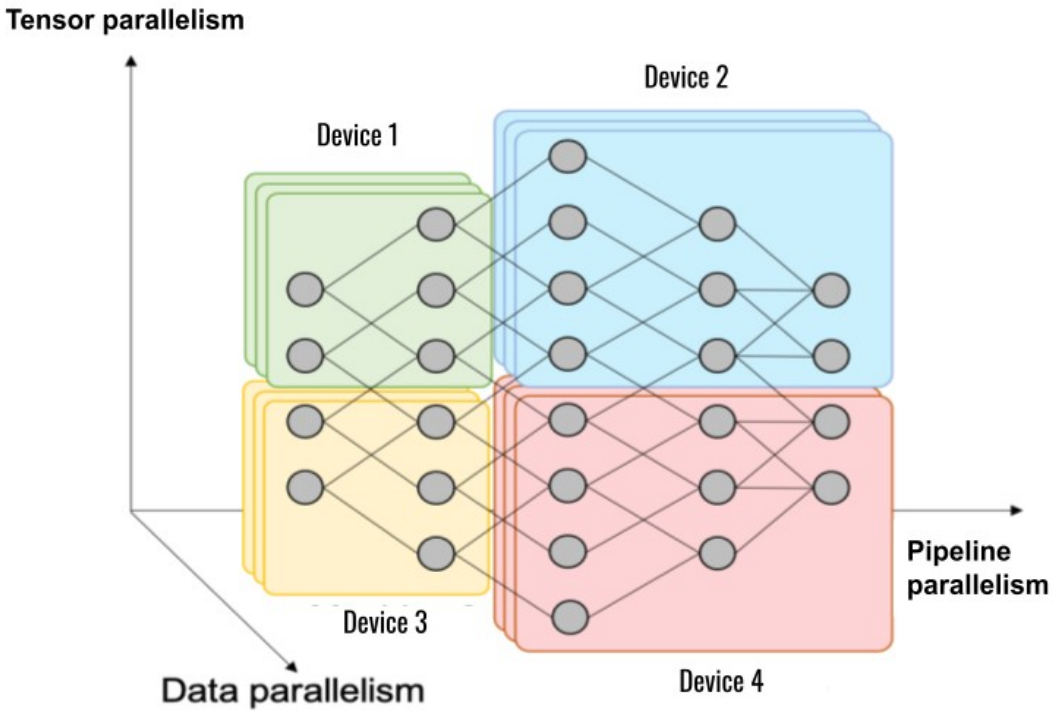
Drawback: Tensor parallelism requires synchronization per layer

TP : Two Layers, One Synchronization



By alternating column and row parallelism, we only need to synchronize once every two layers

3D Parallelism



- **Data Parallelism**
  - Easy to implement
  - Better performance
  - Increases batch size (convergence issue)
- **Pipeline Parallelism**
  - Difficult to implement
  - Balance between memory, performance, and convergence
- **Tensor Parallelism (or FSDP)**
  - Difficult to implement
  - **High bandwidth demand** (Intra-node implementation)

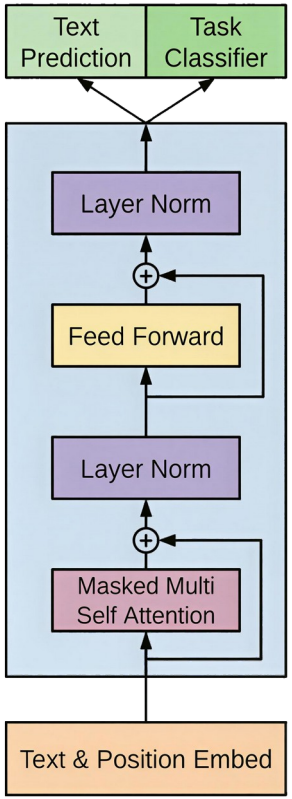
Optimizing the learning of Large Language Model

Context parallelism ◂

Sequence parallelism ◂

Expert parallelism ◂

Transformer Block



Sequence Parallelism

-Utilize Tensor and Pipeline Parallelism

Context Parallelism

-Complexity depends on the number of tokens
-Processing 100 million tokens can requires over 1000GB

CP: Blockwise Parallel Transformer

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

$$= \frac{\exp\left(\frac{QK^T}{\sqrt{d_k}}\right)V}{\sum \exp\left(\frac{QK^T}{\sqrt{d_k}}\right)}$$

Normalization constant

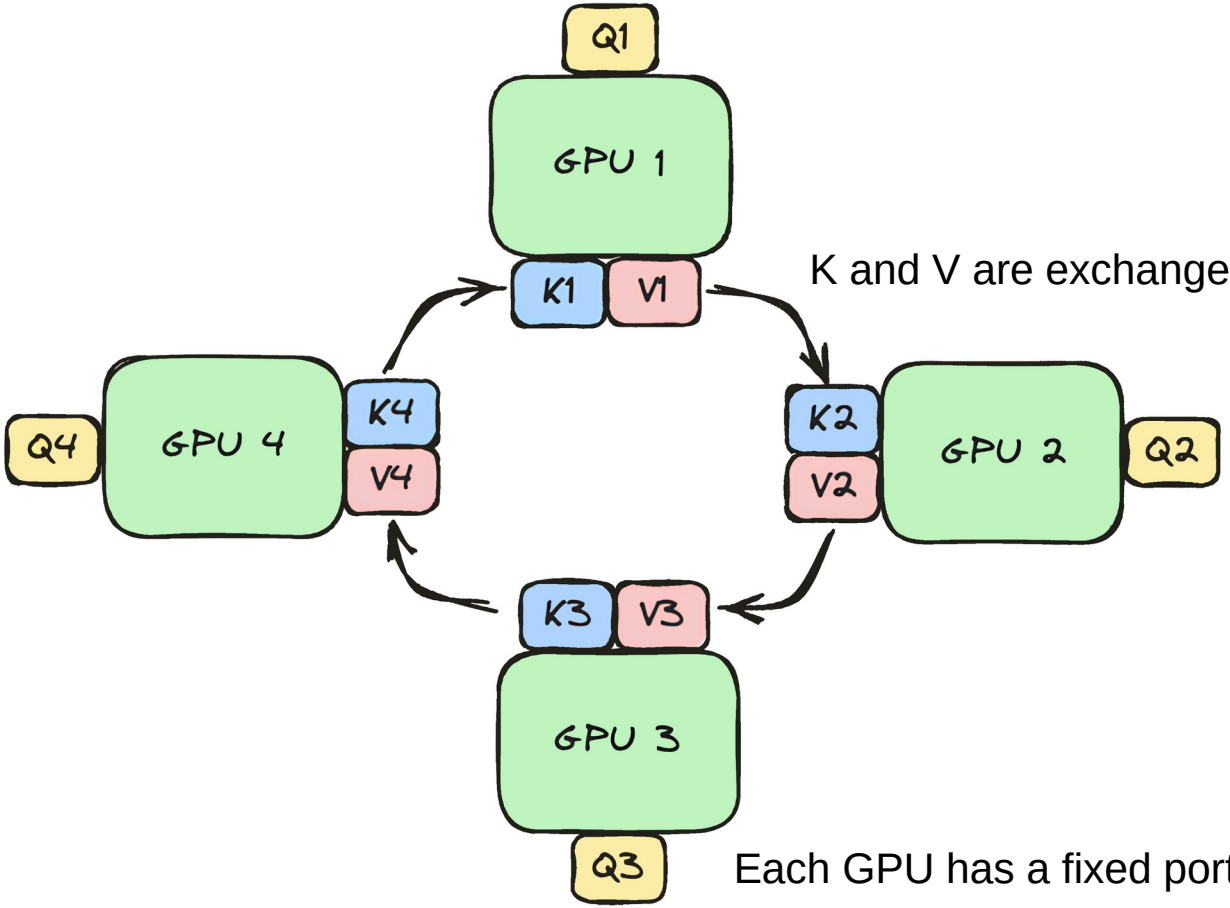
$$\left[\begin{array}{c|c|c} K_{11} & K_{12} & K_{13} \\ K_{21} & K_{22} & K_{23} \end{array}\right] \qquad \left[\begin{array}{c|c} V_{11} & V_{12} \\ V_{21} & V_{22} \\ V_{31} & V_{32} \end{array}\right]$$

$$\left[\begin{array}{c} Q_{11} \quad Q_{12} \\ \hline Q_{21} \quad Q_{22} \\ \hline Q_{31} \quad Q_{32} \end{array}\right] \left[\begin{array}{ccc} QK^t_{11} & QK^t_{12} & QK^t_{13} \\ QK^t_{21} & QK^t_{22} & QK^t_{23} \\ QK^t_{31} & QK^t_{32} & QK^t_{33} \end{array}\right] \xrightarrow{\text{Exp}} \left[\begin{array}{cc} A_{11} & A_{12} \\ \hline A_{21} & A_{22} \\ \hline A_{31} & A_{32} \end{array}\right]$$

⚠ Attention is not normalized

If I want the first row of the attention matrix, I just need the first row of Q

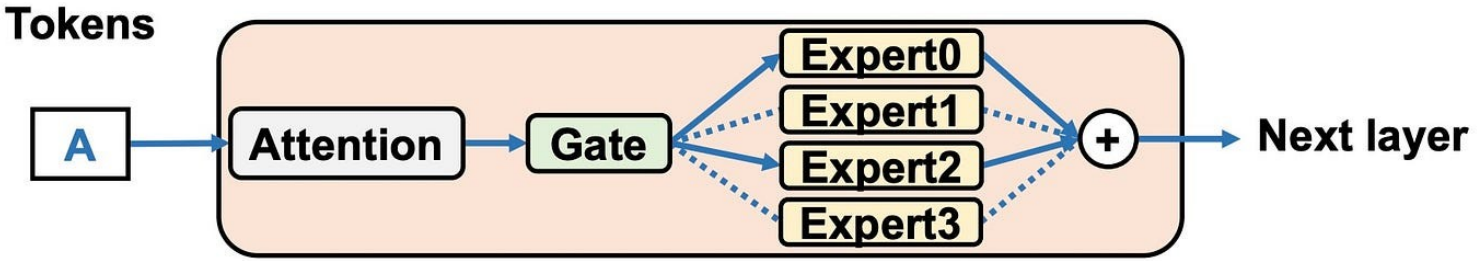
CP: Ring Attention



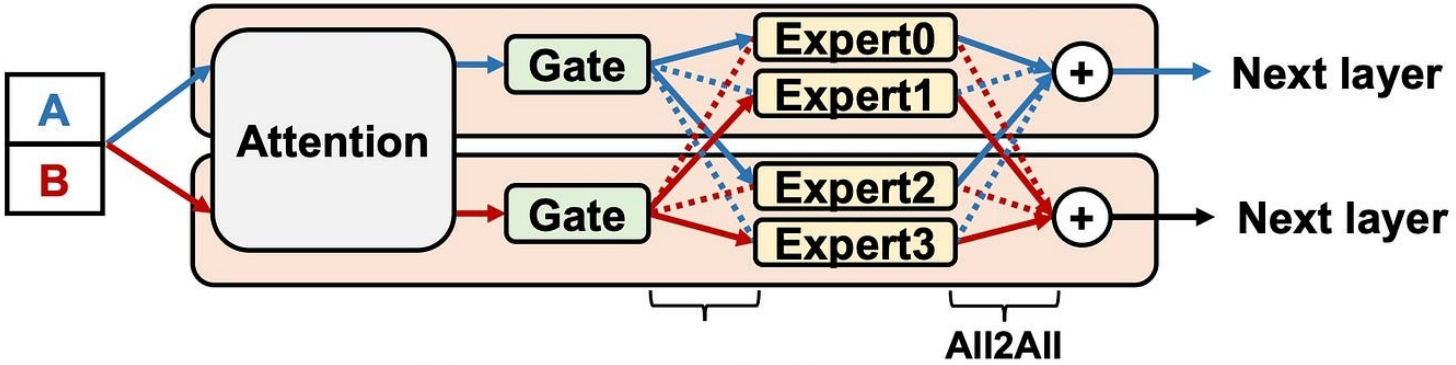
K and V are exchanged between GPUs

Each GPU has a fixed portion of Q

Expert Parallelism

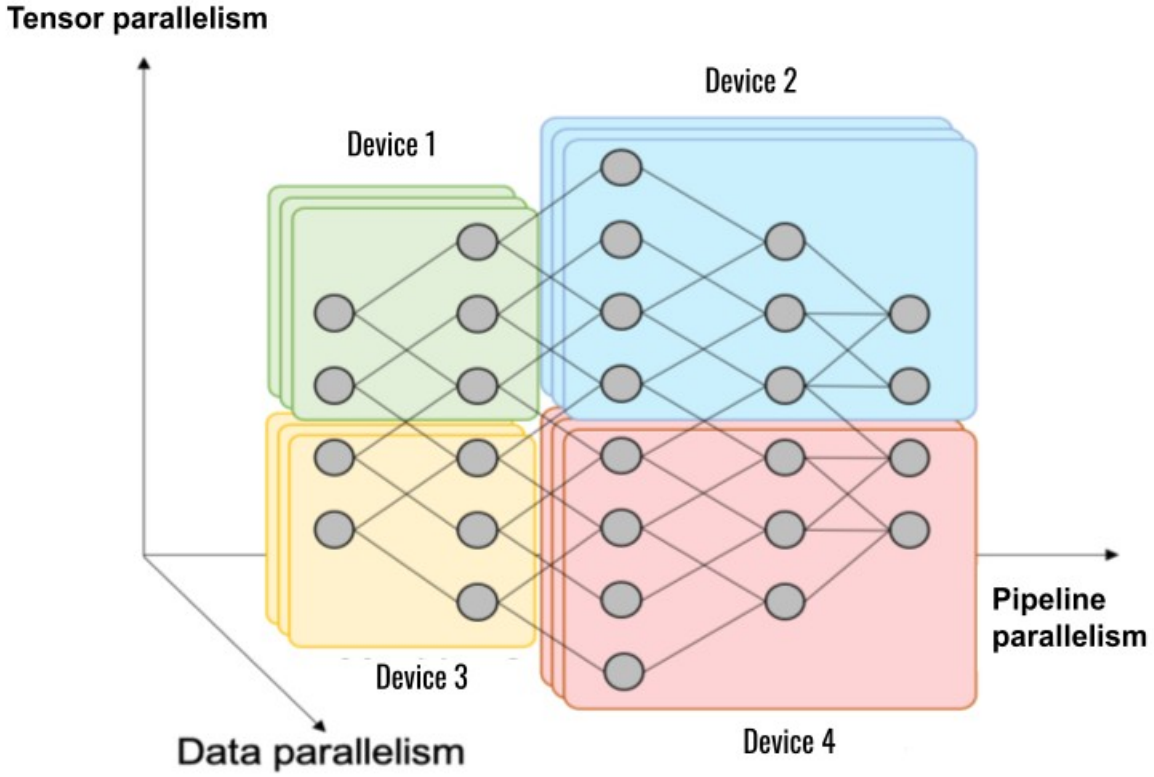


(a) MoE layer.



(b) Expert parallelism.

5D Parallelism



- Data Parallelism
- Pipeline Parallelism
- Tensor Parallelism
- Context Parallelism
- Expert Parallelism